Force-Conditioned Visuomotor Policy Learning through Human Demonstration of Tool Use

Abstract— **Contact-rich manipulation requires reasoning about force, not just trajectory, yet standard imitation learning predicts pose alone. We introduce a method for recovering time-varying contact force from vision-only human demonstrations via robot replay. To improve robustness to camera placement during data collection, we prepare the recorded images by novel view synthesis using 3D Gaussian Splatting. Together with the recorded end-effector pose and the recovered contact force, a diffusion-based visuomotor policy is trained, which jointly generates desired pose and force for deployment on the robot through standard admittance control. Demonstration on a pasta scooping-and-transfer task shows that our method achieves a higher success rate compared with the pose-only baseline.**

I. Introduction

Robots performing contact-rich manipulation tasks — assembly, wiping, assistive cooking/feeding — must reason not only about where to move, but about how much force to apply while doing so. Applying too little force can cause a task to fail outright, while applying too much can damage the tool, the environment, or, in physically assistive settings, pose a safety risk to a person receiving care. For contact-rich manipulation, it is necessary to consider appropriate contact force when applying learning-based control policy.

Recent work has substantially reduced the cost of collecting manipulation demonstrations by removing the need for a robot or any instrumentation at demonstration time. Tool-as-Interface [1] lets a human demonstrate a task with a handheld tool in front of a camera, tracking its motion visually and later converting it into a robot-executable trajectory, eliminating the teleoperation rig or VR headset otherwise required. This makes large-scale demonstration collection far more accessible, but it recovers only *motion* — the tool's pose trajectory — with no mechanism for capturing how much force the human applied. A policy trained on this motion alone, deployed through ordinary position control, inherits this gap: it has no way to know, or enforce, an appropriate contact force at any point in the task.

One might expect this gap to be closed by existing force-control techniques: impedance and admittance control are textbook techniques [2], already exposed as built-in control modes on modern collaborative arms [3]. Admittance control in particular lets a robot yield to external force via a programmable virtual mechanical response — inertia, damping, and stiffness — rather than rigidly holding a commanded position. However, its classical formulation targets a *constant* reference force, suited to tasks like surface polishing where the desired contact force does not vary. Many demonstration-learned tasks do not fit this mold: scooping food, for instance, requires near-zero force during approaching-stage, rising force as the tool engages the food, and further variation with food consistency and fill level. A single hand-tuned reference cannot capture this; what is needed is a reference force that varies with the task, the way a human naturally modulates grip and push force without thinking about it.

In this article, we study how to incorporate time-varying contact force into visuomotor policy learning for contact-rich manipulation. We recover contact force from vision-only human demonstrations by replaying the tracked motion on the robot and estimating the external wrench from onboard joint torque sensing. Together with image data synthesized through 3D Gaussion Splatting process, we train a visuomotor diffusion policy to jointly generate target pose and wrench, and deploy it by supplying the generated pose and wrench as a time-varying setpoint to a standard admittance controller. We demonstrate the developed method on a pasta scooping-and-transfer task, and show that our method improves success rate over a pose-only baseline, and the robot generates contact forces that are closer to the human demonstration reference.

In summary, this paper offers:

1) A method for recovering time-varying target contact force from vision-only human demonstrations, via robot replay and the robot's own onboard joint-torque sensing — requiring no wrist force/torque sensor and no instrumentation on the human demonstrator.
2) A visuomotor policy that jointly predicts a reference pose and a reference wrench at each timestep, trained on this recovered data.
3) A policy deployment scheme in which the predicted wrench serves as a time-varying setpoint for standard admittance control.

II. Related Work

Imitation learning for manipulation: Recent advances in imitation learning have enabled robots to acquire complex manipulation behaviors directly from demonstration, without hand-engineered controllers.

Behavior cloning methods based on action chunking with transformers [4], implicit energy-based policies [5], and diffusion-based policies [6] have proven particularly effective at capturing the multimodal, non-deterministic nature of human demonstration data. More recently, flow-matching [7] and generalist policies [8] have extended this paradigm across tasks.

Demonstration collection: A parallel line of work reduces the cost and instrumentation burden of collecting demonstrations. Tool-as-Interface [1] lets a human demonstrate a task with a handheld tool, with no robot present; the tool's motion is tracked visually and later converted into a robot-executable trajectory. The Universal Manipulation Interface (UMI) [9] similarly collects demonstrations with a handheld gripper decoupled from any robot embodiment, and low-cost teleoperation platforms [10], [11] further lower the barrier to large-scale collection. These methods substantially cut demonstration cost, requiring no teleoperation rig or robot time at demonstration collection — but they recover only motion, not contact force.

Force-conditioned policy learning: A separate line of work has sought to incorporate force into learned policies. Reactive Diffusion Policy (RDP) [12] and its extension ImplicitRDP [13] both find that naively appending force to the policy's observation is largely ineffective — a diffusion policy given force as a passive observation channel performs comparably to a vision-only baseline. Both works show that force is effective when incorporated into the action space: RDP via a fast, force-reactive decoder operating alongside a slower latent policy, and ImplicitRDP via a virtual-target formulation derived from quasi-static compliance control. Outside the imitation-learning setting, reinforcement learning approaches have also demonstrated learned force control on position-interface robots [14]. Across these literatures, the consistent finding is that force is best incorporated at the action or control level, compared to an incidental observation.

Robot-assisted feeding: Assistive feeding is a long-studied application of manipulation research, encompassing the role of force in fork-based bite acquisition [15], adaptive strategies for food properties and user preferences [16], and general-purpose feeding systems [17]. These tasks are contact-rich and safety-critical — excessive force can damage utensils, spill food, or endanger a nearby user — making feeding a natural, high-stakes testbed for force-conditioned imitation learning, and motivating the pasta scooping-and-transfer task studied here.

These two lines of work have progressed independently: methods that eliminate instrumentation don't incorporate force, and methods that incorporate force still require a dedicated wrist force/torque sensor. However, many modern arms already have onboard joint-torque sensing [3], yet no force-conditioned method exploits it to recover force from an instrumentation-free demonstration. This paper sits at that intersection, recovering a time-varying wrench from the robot's own torque sensors and predicting it alongside pose, placing force in the action space as prior work suggests — while also keeping wrench in the observation, as a signal of the current contact state that helps the policy decide what force to apply next.

III. The Robot & Problem Formulation

A. The Robot

We use a Kinova Jaco^2 6-DOF robotic arm with a spherical wrist configuration. Each of the six joints is driven by modular Kinova actuators equipped with integrated torque sensors, in addition to position encoders, enabling joint torque to be read directly at every joint. The robot's end-effector is fitted with a 3D-printed mount that rigidly attaches a 3D-printed spoon, ensuring a rigid attachment between the robot end-effector and the spoon.

The dynamics of the robot arm is described as:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) + \mathbf{J}(\mathbf{q})^\top \mathbf{F}^{ext} = \boldsymbol{\tau}(t) \qquad (1)$$

where $\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}$ are the joint position, velocity, and acceleration, respectively; $\mathbf{M}(\mathbf{q})$, $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}$, and $\mathbf{G}(\mathbf{q})$ are, respectively, the joint-space inertia, Coriolis/centrifugal, and gravitational torque terms arising from the robot's own dynamics; $\mathbf{J}(\mathbf{q})$ is the manipulator Jacobian; $\mathbf{F}^{ext} \in \mathbb{R}^6$ is the external wrench acting at end-effector; and $\boldsymbol{\tau}(t)$ is the joint torque control input to the robot arm.

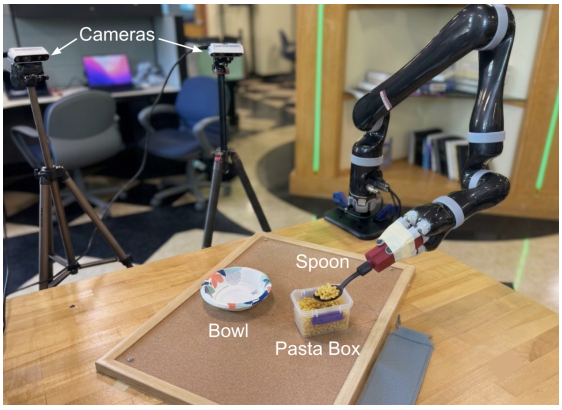


Fig. 1: The experimental setup consists of Kinova robot arm with its end-effector holding a spoon, two RGB-D cameras, and pasta box and bowl for transporting task.

B. The Experimental Setup

The experimental scene contains a pasta box, a bowl, and the robotic arm, as shown in Fig. 1. Two Intel RealSense RGB-D cameras are positioned so that both point at the scene, with the angular separation between their optical axes kept under 90°.

The robot is tasked with scooping pasta from the pasta box and transferring pasta to the nearby bowl. Given human demonstration of the task, recorded by

the cameras, the problem studied in this paper is to control the robot arm to autonomously scoop pasta from the pasta box and place it in the nearby bowl, applying an appropriate amount of contact force throughout the scooping, transferring, and dropping phases of the task — rather than following a trajectory blind to how hard it is pressing — learning this behavior entirely from the demonstration, without any hand-specified force target. We formulate our visuomotor learning problem below.

C. Learning Problem Formulation

We formulate the task as a Markov Decision Process (MDP), and aim to learn a policy π that enables the robot to perform such given task: scooping and transferring pasta.

Observation space: At each timestep t, the robot's observation $\mathbf{o}_t \in O$ consists of three modalities: a single-view RGB image $\mathbf{I}_t \in \mathcal{I}$, the robot's end-effector pose $\mathbf{X}_t \in SE(3)$, and the sensed external wrench at the end-effector $\mathbf{F}_t^{ext} \in \mathbb{R}^6$. Here $\mathbf{F}_t^{ext} = [F_x, F_y, F_z, M_x, M_y, M_z]^\top$ denotes the full six-dimensional wrench, comprising the three force and three moment components at the end-effector. Each image $\mathbf{I}_t$ is a tensor in $\mathbb{R}^{128\times128\times3}$. As standard in visuomotor imitation learning, the policy conditions on a short history of these observations rather than a single timestep. We denote the observation horizon T_o, such that the full conditioning input is

$$\mathbf{o}_t = \left\{\left(\mathbf{I}_\tau,\ \mathbf{X}_\tau,\ \mathbf{F}_\tau^{ext}\right)\ :\ \tau \in [t - T_o + 1,\ t]\right\}. \tag{2}$$

Action space: The action space $\mathcal{A}$ consists of a future chunk of actions over an action horizon T_a. Each action $\mathbf{a}_t \in \mathcal{A}$ consists of a target end-effector pose $\mathbf{X}^r \in SE(3)$ and a target contact wrench $\mathbf{F}^r \in \mathbb{R}^6$:

$$\mathbf{a}_t = \{(\mathbf{X}_\tau^r,\ \mathbf{F}_\tau^r)\ :\ \tau \in [t,\ t + T_a - 1]\}. \tag{3}$$

We refer to $\mathbf{X}^r$ and $\mathbf{F}^r$ as *reference pose* and *reference wrench*, since they are not commands sent directly to the robot's controller, but are instead supplied as the time-varying reference to the admittance correction.

Visuomotor Policy Learning Problem: Given a dataset $\mathcal{D}$ of human demonstrations of the tool use, our goal is to train a policy $\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t)$ that, given an observation $\mathbf{o}_t \in O$, generates an action $\mathbf{a}_t \in \mathcal{A}$, determining the commanded joint motion and the corresponding torque the robot needs to execute it.

IV. Main Method

A. Overview

Our method begins with a human demonstrating the task in front of the cameras using a tool, whose motion we track. This motion is then replayed on the robot, as shown in Fig. 2. While replaying, the robot records its joint torques and, using an estimate of its own dynamics, extracts the external force acting on the tool. We then generate novel views of the scene using 3D Gaussian Splatting, so that the resulting policy is robust to camera viewpoint. Finally, we train a diffusion policy on the recovered pose and force trajectories and deploy it in closed loop, with its predicted wrench supplied as a time-varying setpoint to an admittance controller. In the next subsections, we explain the detailed methods for collecting view-only human demonstrations, recovering force, policy training, and closed-loop control.

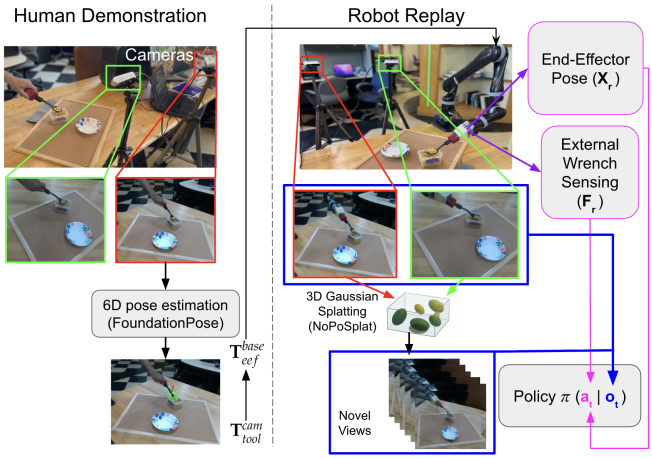


Fig. 2: System overview.

B. Human Demonstration Collection

Human demonstrations are collected without the robot present, and without any instrumentation on the human demonstrator. A human performs the pasta scooping-and-transfer task using a standard spoon, in front of one of the two fixed cameras described in Section III-A. We use FoundationPose [18] to track the 6D pose of the spoon at every frame, yielding the tool's pose relative to the camera frame, $\mathbf{T}_{tool}^{cam}(t) \in SE(3)$.

To express this tracked motion in a frame shared with the robot, we introduce a fixed ChArUco board as a common task frame. The transform from the camera frame to the task frame is $\mathbf{T}_{cam}^{task}$. The tool's pose in the task frame is then obtained by composing the two transforms:

$$\mathbf{T}_{tool}^{task}(t) = \mathbf{T}_{cam}^{task}\,\mathbf{T}_{tool}^{cam}(t). \tag{4}$$

This yields $\mathbf{T}_{tool}^{task}(t)$, the demonstrated tool trajectory expressed in a frame common to both the human demonstration and the robot.

C. Robot Replay & Force Recovery

In this step, the robot is separately calibrated with respect to the same ChArUco board, giving a fixed transform $\mathbf{T}_{task}^{base}$ from the task frame to the robot's base frame as shown in Fig. 3. Composing this with the $\mathbf{T}_{tool}^{task}(t)$, we get the tool motion directly in the robot's base frame:

$$\mathbf{T}_{tool}^{base}(t) = \mathbf{T}_{task}^{base}\,\mathbf{T}_{tool}^{task}(t). \tag{5}$$

To determine the corresponding target pose for the robot's end-effector, we account for the fixed rigid transform $\mathbf{T}_{tool}^{eef}$ between the robot's end-effector and the

tool it holds — identical in geometry to how the human holds the tool — such that:

$$\mathbf{T}_{eef}^{base}(t) = \mathbf{T}_{tool}^{base}(t)\left(\mathbf{T}_{tool}^{eef}\right)^{-1}. \tag{6}$$

This yields $\mathbf{T}_{eef}^{base}(t)$, the desired end-effector pose trajectory the robot must follow to replicate the human-demonstrated tool motion. We solve for the corresponding joint-space trajectory via inverse kinematics:

$$\mathbf{q}_r(t) = \mathrm{IK}\left(\mathbf{T}_{eef}^{base}(t)\right). \tag{7}$$

The robot executes $\mathbf{q}_r(t)$ using position control, so that it strictly retraces the demonstrated trajectory regardless of any contact force encountered along the way. As the robot moves, we record the camera frames ($\mathbf{I}_t^{cam0}$ and $\mathbf{I}_t^{cam1}$), as well as joint torques $\boldsymbol{\tau}(t)$ at every timestep using the onboard torque sensor in each actuator.

The measured torque $\boldsymbol{\tau}(t)$ reflects both the torque required to move the robot's own body and any torque arising from external contact (1). We estimate the gravitational, Coriolis, and inertial torque terms from the measured joint trajectory (see details in Appendix I), and subtract them from the measured torque $\boldsymbol{\tau}(t)$. The residual is attributed to the external contact, and is converted into the corresponding end-effector wrench via the Jacobian pseudoinverse:

$$\mathbf{F}^r(t) = (\mathbf{J}(\mathbf{q})^\top)^+\left(\boldsymbol{\tau}(t) - \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} - \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} - \mathbf{G}(\mathbf{q})\right). \tag{8}$$

This recovers $\mathbf{F}^r(t)$ without any wrist force/torque sensor or instrumentation on the demonstrator. Together, $\left(\mathbf{q}_r(t), \mathbf{F}^r(t)\right)$ — or equivalently $\left(\mathbf{X}^r(t), \mathbf{F}^r(t)\right)$ after forward kinematics — constitute the training labels for the policy.

Note that this step also differs from Tool-as-Interface [1] in how training images are obtained. They train on human demonstration images, masking out the human hand and, at deployment, the robot arm, to keep training and deployment distributions aligned — but segmentation-based masking is not reliably accurate across all environments, and inaccurate masks can degrade deployment performance. Because our training images come directly from robot replay rather than human demonstration, no masking is required at any stage: the robot is simply present in both training and deployment images alike.

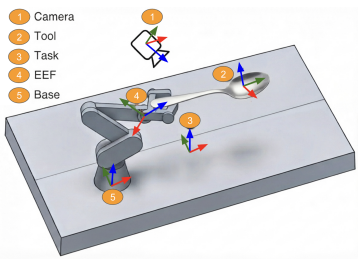


Fig. 3: Coordinate frames used in this work: the camera frame, the tool frame, the task frame, the robot end-effector (EEF) frame, and the robot base frame.

D. Novel View Synthesis

Tool-as-Interface [1] synthesizes novel training viewpoints via a two-stage pipeline: MASt3R [19] reconstructs a 3D point cloud from the two RGB camera views, after which 3D Gaussian Splatting renders novel views from this reconstruction. The authors report this method exhibits noticeable noise and reduced realism, limiting its usefulness for policy training.

We improve this by replacing it with a single feed-forward, pose-free model, NoPoSplat [20], which predicts 3D Gaussians directly from the stereo image pair in one forward pass. Qualitatively, we observe that this produces novel views with substantially fewer dark spots and distortions than the Tool-as-Interface baseline, yielding cleaner and more realistic renders across the interpolated viewpoint range. The two camera images used for this augmentation are $\mathbf{I}_t^{cam0}$ and $\mathbf{I}_t^{cam1}$, collected during *robot replay*.

Our setup provides only two fixed camera viewpoints per episode, while the policy is deployed with a single camera (see Section IV-G for policy deployment). To improve robustness to camera placement without collecting additional demonstrations, we augment the training set with synthetic viewpoints rendered by NoPoSplat, cropped to the largest hole-free rectangle, and rescaled to native resolution — yielding novel views $\{\mathbf{I}_{t,k}^{novel}\}_{k=1}^{K}$ per replay timestep.

This augmentation is valid because our action labels $\left(\mathbf{X}^r(t), \mathbf{F}^r(t)\right)$ are defined in the robot's base/task frame, not any camera's frame: real and novel-view images at a given timestep therefore share an identical label, letting each timestep pair with several alternate viewpoints at no extra data collection cost. During training, the observation image is drawn at random from $\left\{\mathbf{I}_t^{cam0}, \mathbf{I}_t^{cam1}, \mathbf{I}_{t,1}^{novel}, \ldots, \mathbf{I}_{t,N}^{novel}\right\}$. We describe the neural network architecture and training algorithm next.

E. Neural Network Architecture

With pose and wrench trajectories recovered through robot replay and images augmented via novel view synthesis, we next train our policy. We use a diffusion-based policy, which generates an action by starting from random noise and gradually refining it based on the current observation, where the same observation can have several equally valid actions. Fig. 4 shows this architecture, following Chi et al. [6], extended to include wrench in both the observation and action.

Observation encoding: At each of the T_o observation steps, the image $\mathbf{I}$ — drawn from the real and novel-view images as described in Section IV-D — is passed through a shared ResNet-18 encoder [21] (ImageNet-pretrained); the resulting visual feature is concatenated with the end-effector pose $\mathbf{X}$ and the sensed external wrench $\mathbf{F}^{ext}$ for that step. This concatenated vector is

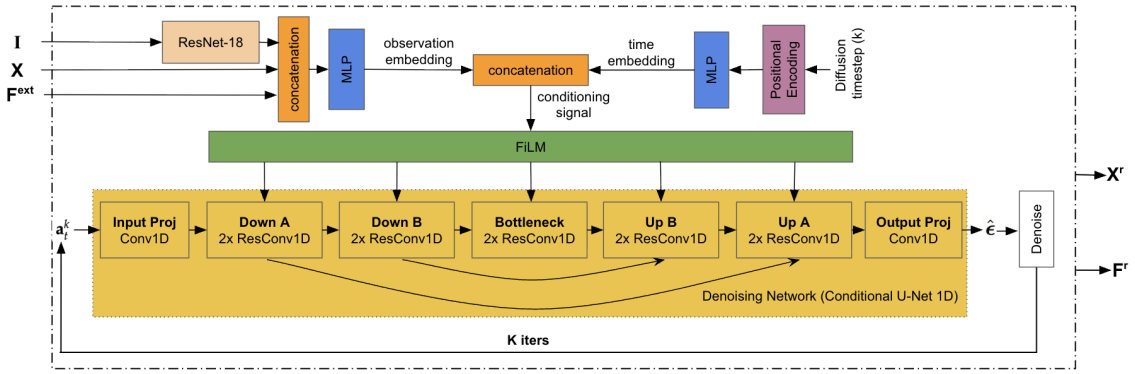


Fig. 4: Neural network architecture.

passed through an MLP (Multi Layer Perceptron) to produce a per-step observation embedding. The embeddings from all T_o steps are then concatenated to form the full observation embedding used for conditioning.

Conditioning signal: In parallel, the current diffusion timestep is encoded via a sinusoidal positional encoding and passed through an MLP to produce a time embedding. The observation embedding and time embedding are concatenated to form a conditioning signal, which is broadcast to every stage of the denoising network via FiLM [22].

Denoising network: The noise-prediction network is a Conditional U-Net 1D [23] operating on the action chunk: an input projection (Conv1D), two downsampling stages (Down A, Down B), a bottleneck, and two upsampling stages (Up A, Up B), each with two residual Conv1D blocks, followed by an output projection (Conv1D). Skip connections link Down A to Up A and Down B to Up B; all five residual-block stages are modulated by the same FiLM conditioning signal.

The network takes a noisy action and predicts its noise, applied iteratively over k diffusion steps; the final denoised output is mapped to $(\mathbf{X}^r, \mathbf{F}^r)$ over the action horizon T_a, used for deployment.

F. Policy Training

Our training algorithm is stated in Algorithm 1. A ground-truth action chunk, $\mathbf{a}_t^0 = \left(\mathbf{X}^r_{t:t+T_a-1}, \mathbf{F}^r_{t:t+T_a-1}\right) \sim \mathcal{D}$, is corrupted at diffusion step k by adding Gaussian noise $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$, giving a noisy action $\mathbf{a}_t^k$. The noise-prediction network, denoted ϵ_θ, is trained to predict this noise from $(\mathbf{a}_t^k, k, \mathbf{o}_t)$, minimizing the mean-squared error between predicted and true noise:

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{a}_t^0, k, \boldsymbol{\epsilon}}\left[\left\|\boldsymbol{\epsilon} - \epsilon_\theta\left(\mathbf{a}_t^k,\, k,\, \mathbf{o}_t\right)\right\|^2\right]. \tag{9}$$

We use $T_o = 2$ and $T_a = 16$.

G. Policy Deployment

At deployment, the trained policy is executed in closed loop on the robot. We prepare the observation

Algorithm 1 Policy Training

Input: Dataset $\mathcal{D}$ containing, per timestep $\{\mathbf{I}_t^{cam0}, \mathbf{I}_t^{cam1}, \mathbf{I}_{t,1}^{novel}, \ldots, \mathbf{I}_{t,N}^{novel}\}$, $\mathbf{X}^r$, and $\mathbf{F}^r$; noise schedule $\{\bar{\alpha}_k\}_{k=1}^K$; learning rate η
Output: Trained network parameters θ

Initialize θ randomly
repeat
  Sample a minibatch of timesteps t from $\mathcal{D}$
  for each sampled t **do**
    for each $\tau \in [t - T_o + 1, t]$ **do**
      Draw $\mathbf{I}_\tau \sim \{\mathbf{I}_\tau^{cam0}, \mathbf{I}_\tau^{cam1}, \mathbf{I}_{\tau,1:N}^{novel}\}$
    end for
    $\mathbf{o}_t \leftarrow \{(\mathbf{I}_\tau, \mathbf{X}_\tau, \mathbf{F}_\tau^{ext}) : \tau \in [t - T_o + 1, t]\}$
    $\mathbf{a}_t^0 \leftarrow \{(\mathbf{X}_\tau^r, \mathbf{F}_\tau^r) : \tau \in [t, t + T_a - 1]\}$ ▷ recovered labels
    Sample $k \sim \text{Uniform}\{1, \ldots, K\}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$
    $\mathbf{a}_t^k \leftarrow \sqrt{\bar{\alpha}_k}\, \mathbf{a}_t^0 + \sqrt{1 - \bar{\alpha}_k}\, \boldsymbol{\epsilon}$
    $\hat{\boldsymbol{\epsilon}}_t \leftarrow \epsilon_\theta(\mathbf{a}_t^k, k, \mathbf{o}_t)$
  end for
  $\theta \leftarrow \theta - \eta \nabla_\theta \dfrac{1}{|\mathcal{B}|} \displaystyle\sum_{t \in \mathcal{B}} \|\boldsymbol{\epsilon} - \hat{\boldsymbol{\epsilon}}_t\|^2$ ▷ AdamW update
until convergence
return θ

$\mathbf{o}_t$ by reading the observation image $\mathbf{I}_t$ from one of the cameras, the external wrench $\mathbf{F}^{ext}(t)$ and the end-effector pose $\mathbf{X}(t)$ over the observation horizon T_o. At each step, the policy produces a reference pose and reference wrench,

$$\left(\mathbf{X}^r_{t:t+T_a},\ \mathbf{F}^r_{t:t+T_a}\right) = \pi_\theta(\mathbf{o}_t), \tag{10}$$

by iteratively denoising from Gaussian noise through the trained network ϵ_θ over k diffusion steps as shown in Algorithm 2. The reference pose and wrench alone don't tell the robot how to respond when sensed force deviates from prediction. Admittance control [2] provides this coupling: it maps end-effector force error into a pose correction via a virtual spring-damper system, yielding when the robot presses too hard and pressing further when it presses too little — well suited to

Algorithm 2 Force-Aware Policy Deployment

Input: Trained network ϵ_θ; observation $\mathbf{o}_t$ comprising image $\mathbf{I}_t$ (live camera), end-effector pose $\mathbf{X}_t$, and sensed external wrench $\mathbf{F}_t^{ext}$; execution horizon $T_{exec} \le T_a$

Output: Commanded joint torque $\boldsymbol{\tau}(t)$

Initialize $\mathbf{X}^{cmd}$, trajectory buffer, and step counter $c \leftarrow T_{exec}$
loop
  Acquire observation $\mathbf{o}_t$ (2)
  if $c \ge T_{exec}$ **then**
    Sample $\mathbf{a}_t^K \sim \mathcal{N}(0, \mathbf{I})$
    for $k = K, K-1, \ldots, 1$ **do**
      $\hat{\boldsymbol{\epsilon}} \leftarrow \epsilon_\theta(\mathbf{a}_t^k, k, \mathbf{o}_t)$
      $\mathbf{a}_t^{k-1} \leftarrow \text{Denoise}\left(\mathbf{a}_t^k, \hat{\boldsymbol{\epsilon}}\right)$
    end for
    unpack $(\mathbf{X}_{r,\,t:t+T_a}, \mathbf{F}_{r,\,t:t+T_a}) \leftarrow \mathbf{a}_t^0$
    place $(\mathbf{X}_{r,\,t:t+T_a}, \mathbf{F}_{r,\,t:t+T_a})$ into trajectory buffer
    $c \leftarrow 0$
  end if
  $\mathbf{X}^r(t), \mathbf{F}^r(t) \leftarrow$ interpolate trajectory buffer
  Compute $\mathbf{F}^{ext}(t)$
  Compute $\mathbf{e}(t) = \mathbf{F}^{ext}(t) - \mathbf{F}^r(t)$
  $\mathbf{X}^{cmd}(t) \leftarrow \mathbf{X}^r(t) + \mathbf{e}(t)$
  $\mathbf{q}_{cmd}(t) \leftarrow \text{IK}\left(\mathbf{X}^{cmd}(t)\right)$
  Send $\mathbf{q}_{cmd}(t)$ to the position controller
  Controller computes $\boldsymbol{\tau}(t)$ to move the robot
  $c \leftarrow c + 1$
end loop

contact-rich tasks like pasta scooping, where pose alone cannot capture appropriate contact behavior.

Admittance correction: Admittance relates the deviation between a reference and an actual pose to a force error via a virtual mass-spring-damper model:

$$\boldsymbol{\Lambda}_d \ddot{\mathbf{e}} + \mathbf{D}_d \dot{\mathbf{e}} + \mathbf{K}_d \mathbf{e} = \mathbf{F}^{ext} - \mathbf{F}^r, \qquad \mathbf{e} = \mathbf{X} - \mathbf{X}^r, \tag{11}$$

where $\boldsymbol{\Lambda}_d$, $\mathbf{D}_d$, and $\mathbf{K}_d$ are the virtual inertia, damping, and stiffness matrices; $\mathbf{X}$ and $\mathbf{X}^r$ are the actual and reference end-effector pose, respectively, with $\mathbf{e}$ their deviation; and $\mathbf{F}^r$ is the reference wrench tracked against the sensed external wrench $\mathbf{F}^{ext}$. The predicted reference pose $\mathbf{X}^r(t)$ and reference wrench $\mathbf{F}^r(t)$, together with the live sensed wrench $\mathbf{F}^{ext}(t)$, are supplied to this admittance correction. Solving (11) for $\mathbf{e}(t)$ gives the corrected pose

$$\mathbf{X}^{cmd}(t) = \mathbf{X}^r(t) + \mathbf{e}(t). \tag{12}$$

$\mathbf{X}^{cmd}(t)$ is converted to a commanded joint position via inverse kinematics,

$$\mathbf{q}_{cmd}(t) = \text{IK}\left(\mathbf{X}^{cmd}(t)\right), \tag{13}$$

and sent to the robot's position controller, which computes the joint torque $\boldsymbol{\tau}(t)$ required to drive the measured joint position $\mathbf{q}(t)$ toward $\mathbf{q}_{cmd}(t)$.

V. Experimental Results

A. Experimental Setup

We collected 33 human demonstrations of the pasta scooping-and-transfer task, each lasting approximately 10 seconds. Throughout data collection, the location of the pasta box and the bowl was varied across episodes to introduce spatial diversity into the demonstrations. Each demonstration was then replayed on the robot following the procedure described in Section IV-C; in 3 of the 33 episodes, tracking error in the tool pose estimate caused the robot to spill pasta during replay. We discarded these episodes and used the remaining 30 clean episodes to train the policies evaluated in this paper.

We evaluate performance based on success rate across the various conditions studied, each evaluated over 40 trials. We define a trial as successful if the robot scoops pasta from the pasta box and transfers it into the bowl without any spilling.

B. Ablation Study

We ablate two components of our pipeline: whether wrench is used alongside pose, and how the training data is collected. Both ablations hold the policy architecture (diffusion) fixed. Table I reports the results. Comparing *Kinova Joystick Teleoperation* against *Ours* isolates the contribution of our replay-based wrench recovery: both predict pose and wrench with admittance deployment, differing only in data collection. Replacing human demonstration and replay with direct joystick teleoperation — recording pose and force live — collapses success to zero, since mapping a low-dimensional joystick to precise 6-DOF tool motion is intrinsically difficult and yields poor demonstrations for fine, contact-rich manipulation.

Comparing *Pose-only* against *Ours* isolates the contribution of force itself: both use identical demonstration and replay data, differing only in whether wrench and admittance deployment are used. Success drops to 82.5% without force, confirming force contributes meaningfully beyond pose alone. Together, these ablations show our pipeline's benefit comes from naturalistic demonstration and replay-based recovery together — neither force nor data collection method alone is sufficient.

C. Policy Architecture Comparison

To test whether our pose and wrench based formulation is specific to the diffusion policy used elsewhere in this paper, we train the same formulation using ACT [4] and flow-matching [7] in place of diffusion. Table II reports success rate for each. ACT and flow-matching achieve comparable success (92.5% each) to our diffusion-based policy (97.5%) when trained on identical recovered data. This indicates our pipeline's benefit is not tied to a specific policy architecture, while diffusion achieves the best performance.

TABLE I: Ablation of force usage and data collection method (40 trials each).

Condition	Data Collection	Wrench	Success Rate
Joystick teleoperation	Joystick	✓	0%
Pose-only (no wrench)	Human Demo + Replay	×	82.5%
Ours	Human Demo + Replay	✓	**97.5%**

TABLE II: Policy architecture comparison, all trained on identical pose+wrench data (40 trials each).

Architecture	Success Rate
ACT [4]	92.5%
Flow Matching [7]	92.5%
Ours (Diffusion [6])	**97.5%**

TABLE III: Comparison with the Tool-as-Interface baseline (40 trials each).

Method	Success rate
Baseline [1]	80%
Ours	**97.5%**

D. Comparison with Baseline

We compare our method against Tool-as-Interface [1] as a baseline, which uses only human demonstration — no robot replay, no force — training directly on demonstration images and generating pose only. Since the human hand is present in training images but absent at deployment, the original method masks the hand out during training and the robot arm out at deployment to keep the two distributions aligned; we reimplement this using a UNet-based lightweight segmentation model [23] for faster masking.

Table III reports success rate for both methods. Our method outperforms the baseline by a wide margin. Beyond the benefit of force (Section V-B), we attribute part of this gap to masking itself: segmentation-based masks are not reliably accurate across environments, and inaccurate masks at deployment can degrade performance. Since our training images come from robot replay rather than human demonstration, our method needs no masking at all, avoiding this failure mode entirely.

The baseline's novel view synthesis strategy is a separate component we improve upon. Fig. 5 compares views rendered by MASt3R [19] + 3D Gaussian Splatting against NoPoSplat [20] at the same interpolated viewpoint: the baseline shows severe distortions, dark regions, and largely unrecognizable geometry — consistent with the reduced realism the authors themselves reported, while NoPoSplat produces a clean, realistic render with the scene, tool, and bowl clearly recognizable.

E. Force Comparison

Success rate alone does not indicate whether a policy's physical interaction matches human demonstration — two policies can both succeed while applying very different contact forces. We compare contact-force statistics between a pose-only policy and ours (pose and wrench) against the reference wrench recovered from training data, over the same 40 trials used in Section V-B. Results are in Table IV. Both mean and average peak force under our policy are closer to the training-data reference, consistent with the pose-only policy under-committing to contact. Neither condition approaches the training data's highest peak force (14.91 N), placing both within a safe operating range relative to the human demonstrations.



Fig. 5: Novel view synthesis comparison.

TABLE IV: Contact-force statistics.

	Training data (30 eps)	Pose Only (40 trials)	Ours (40 trials)
Mean $\|\mathbf{F}\|$	4.19 N	3.68 N	4.06 N
Avg. peak $\|\mathbf{F}\|$	9.32 N	6.57 N	7.19 N
Overall max $\|\mathbf{F}\|$	14.91 N	8.11 N	7.69 N

VI. Conclusion

We presented a method for incorporating contact force into visuomotor policy learning without a dedicated force/torque sensor. By tracking a tool during a vision-only human demonstration and replaying the trajectory on the robot, we recover a reference wrench from onboard joint torque sensing, using the robot's own dynamics model to isolate external contact force. This wrench, with the demonstrated pose, supervises a visuomotor policy that jointly predicts reference pose and wrench at deployment, supplied as a time-varying setpoint to an admittance controller. On a pasta scooping-and-transfer task, this achieves a higher success rate than other test cases and remains effective across diffusion, transformer-based (ACT), and flow-matching architectures alike.

References

[1] H. Chen, C. Zhu, S. Liu, Y. Li, and K. R. Driggs-Campbell, "Tool-as-interface: Learning robot policies from observing human tool use," in *Conference on Robot Learning*, 2025.

[2] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017.

[3] F. Boucher, H. Lamontagne, A. Campeau-Lecours, S. Latour, P. Fauteux, V. Maheu, C. Deguire, and L.-J. Caron L'Ecuyer, "Kinova modular robot arms for service robotics applications," 2017.

[4] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Robotics: Science and Systems (RSS)*, 2023.
[5] P. Florence, C. Lynch, A. Zeng, O. A. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson, "Implicit behavioral cloning," in *Conference on robot learning*, 2022.
[6] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, 2025.
[7] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *International Conference on Learning Representations (ICLR)*, 2023.
[8] K. Black, O. Mees, H. Walke, K. Pertsch, D. Ghosh, J. Hejna, T. Kreiman, C. Xu, J. Luo, Y. L. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine, "Octo: An open-source generalist robot policy," in *Robotics: Science and Systems (RSS)*, 2024.
[9] C. Chi, Z. Xu, C. Pan, E. Cousineau, B. Burchfiel, S. Feng, R. Tedrake, and S. Song, "Universal manipulation interface: In-the-wild robot teaching without in-the-wild robots," in *Robotics: Science and Systems (RSS)*, 2024.
[10] Z. Fu, T. Z. Zhao, and C. Finn, "Mobile aloha: Learning bimanual mobile manipulation using low-cost whole-body teleoperation," in *Conference on Robot Learning*, 2024.
[11] P. Wu, Y. Shentu, Z. Yi, X. Lin, and P. Abbeel, "Gello: A general, low-cost, and intuitive teleoperation framework for robot manipulators," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.
[12] H. Xue, J. Ren, W. Chen, G. Zhang, Y. Fang, G. Gu, H. Xu, and C. Lu, "Reactive diffusion policy: Slow-fast visual-tactile policy learning for contact-rich manipulation," in *Robotics: Science and Systems (RSS)*, 2025.
[13] W. Chen, H. Xue, Y. Wang, F. Zhou, J. Lv, Y. Jin, S. Tang, C. Wen, and C. Lu, "Implicitrdp: An end-to-end visual-force diffusion policy with structural slow-fast learning," *IEEE Robotics and Automation Letters*, 2026.
[14] C. C. Beltran-Hernandez, D. Petit, I. G. Ramirez-Alpizar, T. Nishi, S. Kikuchi, T. Matsubara, and K. Harada, "Learning force control for contact-rich manipulation tasks with rigid position-controlled robots," *IEEE Robotics and Automation Letters*, 2020.
[15] T. Bhattacharjee, G. Lee, H. Song, and S. S. Srinivasa, "Towards robotic feeding: Role of haptics in fork-based food manipulation," *IEEE Robotics and Automation Letters*, 2019.
[16] E. K. Gordon, X. Meng, M. Barnes, T. Bhattacharjee, and S. S. Srinivasa, "Adaptive robot-assisted feeding: An online learning framework for acquiring previously unseen food items," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
[17] R. Feng, Y. Kim, G. Lee, E. K. Gordon, M. Schmittle, S. Kumar, T. Bhattacharjee, and S. S. Srinivasa, "Robot-assisted feeding: Generalizing skewering strategies across food items on a plate," in *International Symposium of Robotics Research (ISRR)*. Springer International Publishing, 2022.
[18] B. Wen, W. Yang, J. Kautz, and S. Birchfield, " FoundationPose: Unified 6D Pose Estimation and Tracking of Novel Objects ," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
[19] V. Leroy, Y. Cabon, and J. Revaud, "Grounding image matching in 3d with mast3r," in *European conference on computer vision*, 2024.
[20] B. Ye, S. Liu, H. Xu, X. Li, M. Pollefeys, M.-H. Yang, and S. Peng, "No pose, no problem: Surprisingly simple 3d gaussian splats from sparse unposed images," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 54 009–54 033.
[21] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE conference on computer vision and pattern recognition*, 2016.
[22] E. Perez, F. Strub, H. De Vries, V. Dumoulin, and A. Courville, "Film: Visual reasoning with a general conditioning layer," in *AAAI conference on artificial intelligence*, 2018.
[23] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *International Conference on Medical image computing and computer-assisted intervention*, 2015.
[24] C. G. Atkeson, C. H. An, and J. M. Hollerbach, "Estimation of inertial parameters of manipulator loads and links," *The International Journal of Robotics Research*, 1986.
[25] W. Khalil and E. Dombre, *Modeling, identification and control of robots*. Butterworth-Heinemann, 2004.

Appendix I
Dynamics Parameter Estimation

Recovering the external wrench $\mathbf{F}^{ext}$ from measured joint torque (1) requires estimates of $\mathbf{G}(\mathbf{q})$, $\mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}}$, and $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}}$, so they can be subtracted and the remainder attributed to external contact. We estimate these using the same replay trajectories, executed in two different modes.

Gravity. Gravity torque is linear in a per-link barycentric parameter set $\boldsymbol{\phi} \in \mathbb{R}^{24}$ (mass and mass-weighted center-of-mass, four per link) [24], [25], giving an analytically-derived regressor $\mathbf{Y}_g(\mathbf{q}) \in \mathbb{R}^{6\times 24}$, computed directly from the robot's forward kinematics following the standard construction in [24], [25], such that $\mathbf{G}(\mathbf{q}) = \mathbf{Y}_g(\mathbf{q})\boldsymbol{\phi}$. Since gravity torque depends only on pose $\mathbf{q}$, not on velocity or acceleration, we calibrate $\boldsymbol{\phi}$ by executing the replay trajectories while repeatedly stopping the robot at many points along each trajectory, ensuring it is fully static, and reading the joint torque at each stop. We fit $\boldsymbol{\phi}$ via ridge-regularized least squares over these static readings; using poses drawn from the task's own trajectories, rather than generic randomized poses spread across the whole workspace, produced a substantially better fit exactly where it is needed. A residual gravity error remains after this fit; we correct it with a small neural network $\mathbf{G}_{res}(\mathbf{q})$ (two hidden layers, width 32, ReLU), taking only $[\sin(\mathbf{q}), \cos(\mathbf{q})]$ as input, trained exclusively on this same static data, keeping it decoupled from the velocity/acceleration-dependent terms below.

Mass and Coriolis. The same parametrization extends to $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}}$ [25], now with a 72-dimensional parameter vector $\boldsymbol{\psi}$ (ten inertial parameters per link, two friction parameters per joint) and a corresponding regressor $\mathbf{Y}_{full}(\mathbf{q},\dot{\mathbf{q}},\ddot{\mathbf{q}})$:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} = \mathbf{Y}_{full}(\mathbf{q},\dot{\mathbf{q}},\ddot{\mathbf{q}})\,\boldsymbol{\psi}. \tag{14}$$

Unlike gravity, these terms depend on velocity and acceleration, so we instead execute the same trajectories continuously, without stopping, and record the joint torque throughout the motion. Since this torque already includes the (now-modeled) gravity contribution, we subtract the gravity estimate described above first; the remainder is attributed to mass and Coriolis effects and used to fit $\boldsymbol{\psi}$ via the same ridge-regularized least squares. We use this linear regressor in our final pipeline, as it is simple, numerically stable, and requires no additional training beyond the least-squares fit itself.